

Shortest Path

2026-1학기 부경대학교 알고리즘 스터디

목차

- 최단 경로 알고리즘
 - 정의
- 다익스트라 알고리즘
 - 동작 과정
 - 구현
 - 시간 복잡도
 - 한계

- 플로이드-워셜 알고리즘
 - 점화식
 - 동작 과정
 - 시간 복잡도
- 벨만-포드 알고리즘
 - 점화식
 - 동작 과정
 - 시간 복잡도
- 경로 추적

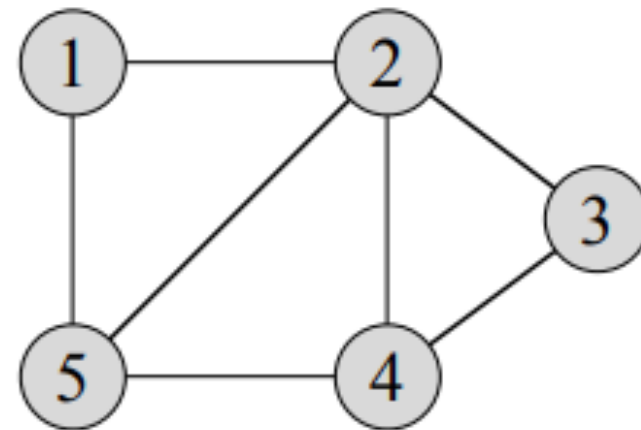
최단 경로 알고리즘

- 경로 (path)란?

- 인접한 정점들이 순서대로 나열된 형태
- 예) $P = (1, 2, 5, 4)$
- 두 정점을 잇는 경로 중 가중치의 합이 가장 작은 경로를 최단 경로라고 함

- 최단 경로 알고리즘이란?

- 두 정점의 최단 경로를 찾는 알고리즘



다익스트라 알고리즘

- 다익스트라 알고리즘이란?

- 가중치가 존재하는 방향 그래프에서 하나의 정점 s 로부터 다른 모든 정점으로의 최단 경로를 구하는 알고리즘

- ※ 다익스트라 알고리즘은 간선의 가중치는 0보다 같거나 클 경우만 성립

- 용어

- S = 최단 거리가 확정된 정점들의 집합

- ※ $S = \emptyset$ 로 초기화

- Q = 최단 거리가 확정되지 않은 정점들의 집합

- $d[i]$ = 정점 s 에서 i 까지의 최단 거리

- ※ $s \rightarrow s$ 는 비용이 발생하지 않으므로 $d[s] = 0$

- ※ 그 외의 정점 i 는 $d[i] = \text{INF}$ 로 초기화

다익스트라 알고리즘

- 동작 과정

- 1. Q에서 d값이 가장 작은 노드 u 를 선택한다
- 2. u 를 S에 추가한다
- 3. u 를 통해 나가는 모든 간선 $w(u, v)$ 에 대해
 - $d[v] = \min(d[v], d[u] + w(u, v))$ 연산을 한다.
 - ※ 즉 노드 v 로 가는 경로 중 u 를 통해서 이동하는게 최단 경로인지 확인한다.
- Q가 공집합이 될 때까지 위 과정을 반복한다.

다익스트라 알고리즘

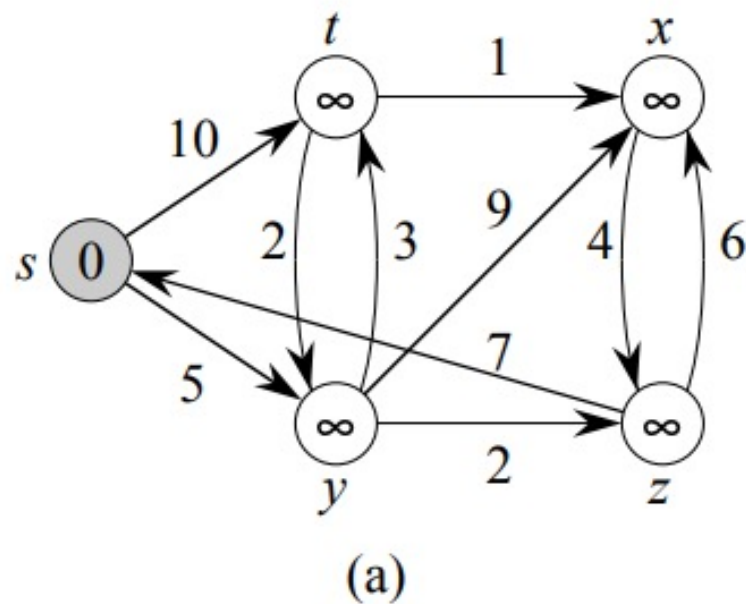
- 예제

- 왼쪽 그림에서 노드 s 에서 시작해 다른 모든 노드의 최단 경로를 구한다고 가정해보자
- 노드 s 로의 최단 거리는 0이므로 $d[s] = 0$ 초기화, 다른 노드들은 아직

최단 경로를 구하지 않았으므로 $d[i] = \text{INF}$ 초기화

$S = \emptyset$

$Q = \{s, t, x, y, z\}$



다익스트라 알고리즘

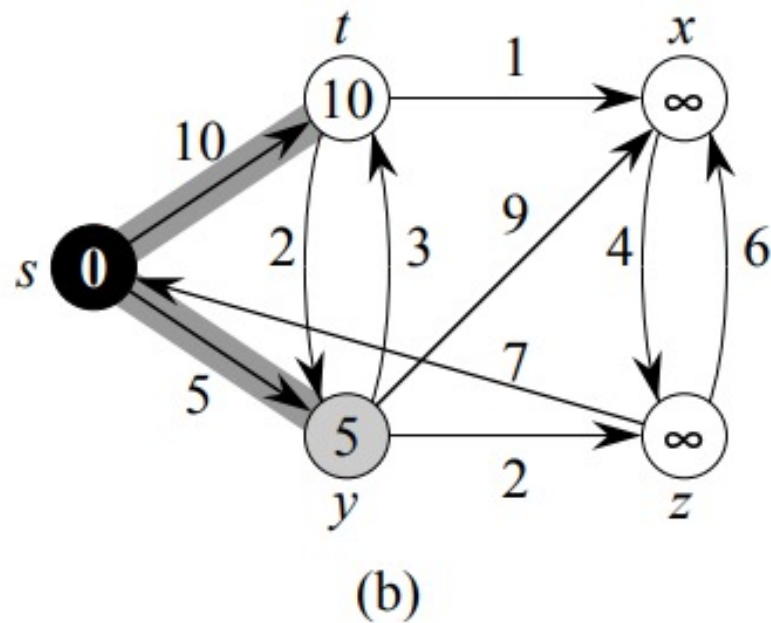
• 예제

- Q에서 d가 가장 작은 s를 골라서 S에 추가
- s로부터 도착하는 노드 t, y에 대하여
- $d[t] = \min(d[t], d[s] + w(s, t))$
- $d[y] = \min(d[y], d[s] + w(s, y))$

※ 두 연산의 결과 $d[t] = 10$, $d[y] = 5$ 가 된다.

$S = \{s\}$

$Q = \{t, y, x, z\}$



다익스트라 알고리즘

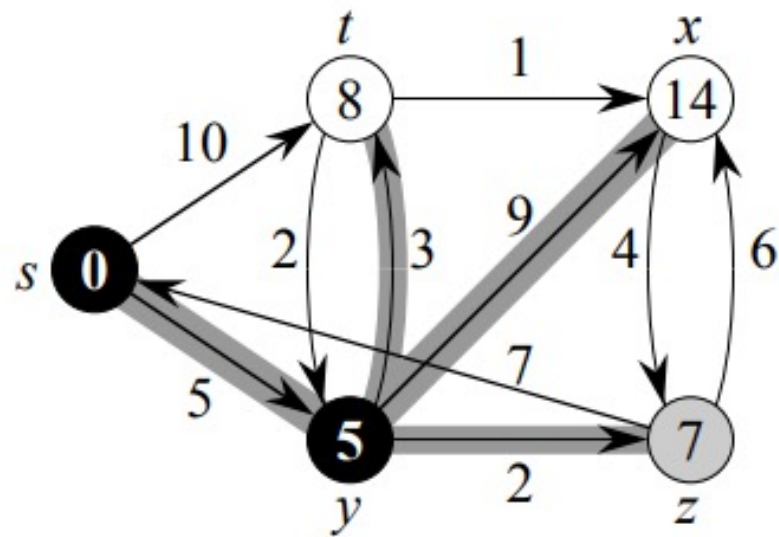
• 예제

- Q에서 d가 가장 작은 y를 골라 s에 추가
- y로부터 도착하는 노드 t, z, x에 대하여
- $d[t] = \min(d[t], d[y] + w(y, t))$
- $d[z] = \min(d[z], d[y] + w(y, z))$
- $d[x] = \min(d[x], d[y] + w(y, x))$

* 연산의 결과 $d[t] = 8$, $d[z] = 7$, $d[x] = 14$ 가 된다.

$$S = \{s, y\}$$

$$Q = \{t, x, z\}$$



(c)

다익스트라 알고리즘

- 예제

- Q에서 d가 가장 작은 z를 골라 s에 추가

- z로부터 도착하는 노드 x, z에 대하여

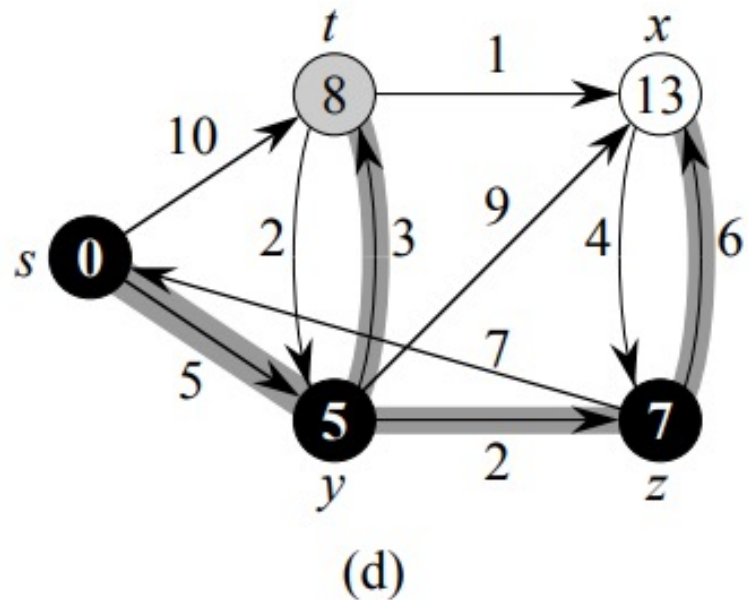
- $d[x] = \min(d[x], d[z] + w(x, z))$

- $d[s] = \min(d[s], d[z] + w(x, s))$

* 두 연산의 결과 $d[x] = 13$ 이 된다. $d[s] = 0$ 이므로 새로 탐색한 경로보다 짧다.

$S = \{s, y, z\}$

$Q = \{t, x\}$

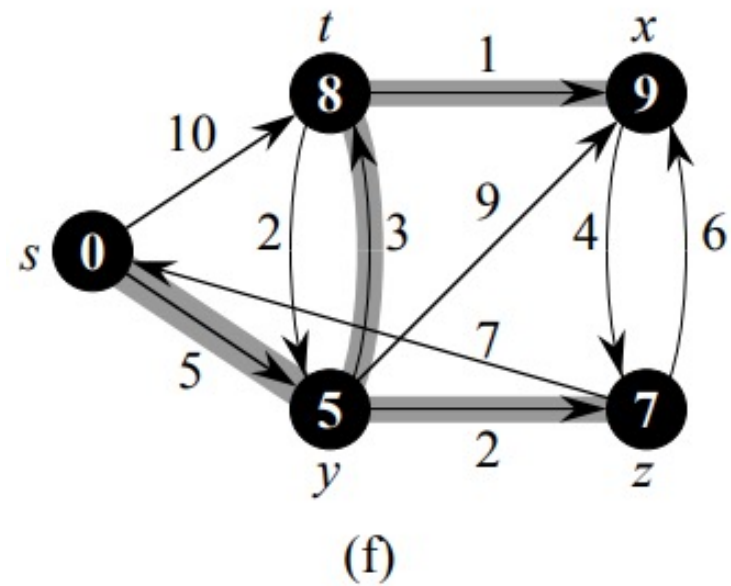


다익스트라 알고리즘

- 예제

- Q가 공집합이 될 때까지 반복하면 그림과 같이 노드 s로부터의 최단 경로를 구할 수 있다

※ 매 반복마다 Q의 원소 하나가 S로 이동하므로 정점의 개수만큼 반복한다.



다익스트라 알고리즘

```
#include <iostream>
#include <algorithm>

#define INF 987654321
using namespace std;

// 예제와 동일
int edge[5][5] = {
    { 0, 10, 5, INF, INF},
    {INF, 0, 2, 1, INF},
    {INF, 3, 0, 9, 2},
    {INF, INF, INF, 0, 4},
    { 7, INF, INF, 6, 0}
};

bool S[5]; // 노드 i가 집합 S에 존재하면 S[i] = true
int d[5]; // 노드 0으로부터의 최단 거리
```

```
// 0번 노드로부터 다른 노드로의 최단 거리를 구하는 다익스트라 알고리즘
void dijkstra() {

    // 최단 거리 초기화
    for(int i = 0; i < 5; i++)
        d[i] = INF;
    d[0] = 0;

    int V = 5; // 정점의 개수

    while(V-- > 0) {

        // 아직 S에 들어가지 않은 노드 중 거리가 가장 작은 노드 찾아서 cur에 대입
        int min_dist = INF, cur = 0;
        for(int i = 0; i < 5; i++) {
            if(!S[i] && d[i] < min_dist) {
                cur = i;
                min_dist = d[cur];
            }
        }

        // 해당 노드 집합 S에 추가
        S[cur] = true;

        // cur과 연결된 모든 노드들에 대해서 최단 거리 업데이트
        for(int nxt = 0; nxt < 5; nxt++) {
            if(edge[cur][nxt] < INF)
                d[nxt] = min(d[nxt], d[cur] + edge[cur][nxt]);
        }
    }
}
```

다익스트라 알고리즘

- 시간 복잡도
 - 선형 탐색으로 d 가 가장 낮은 노드를 선택하므로 $O(V^2)$
 - 우선순위 큐를 사용하면 d 가 낮은 노드를 $O(\log V)$ 만에 찾을 수 있으므로 $O((V + E) \log V)$ 로 최적화 가능

다익스트라 알고리즘

```
void dijkstra() {  
  
    // 현재 거리가 가장 낮은 노드 먼저 살펴봄  
    // first : 현재 가중치 second : 노드  
    priority_queue<pair<int, int> > pq;  
  
    // 최단 거리 초기화  
    for(int i = 0; i < 5; i++)  
        d[i] = INF;  
    d[0] = 0;  
  
    pq.push(make_pair(0, 0));  
  
    while(!pq.empty()) {  
        pair<int, int> cur = pq.top(); pq.pop();  
  
        int w = -cur.first;  
        int node = cur.second;  
  
        // 만약 현재 거리가 d에 저장된 거리보다 크다면 더 탐색 x  
        if(d[node] < w) continue;  
  
        // 가능한 다음 노드들에 대하여  
        for(edge nxt : edges[node]) {  
            int n_node = nxt.b;  
            int n_w = w + nxt.w;  
            // 만약 d에 저장된 거리보다 크다면 pq에 추가 x  
            if(d[n_node] ≤ n_w)  
                continue;  
            d[n_node] = n_w;  
            pq.push(make_pair(-n_w, n_node));  
        }  
    }  
}
```

다익스트라 알고리즘

- 다익스트라 알고리즘의 의의

- 시작 정점에서 다른 모든 정점까지의 최단 거리를 구하는 대표적인 알고리즘
- 가장 비용이 낮은 경로부터 탐색하는 방법은 여러 경로 탐색 알고리즘의 기반이 됨

※ 대표적인 알고리즘으로, A* (다익스트라에 휴리스틱한 거리 추정 비용을 추가하는 알고리즘)가 존재

- 다익스트라 알고리즘의 한계

- 최단 경로가 확정된 정점은 더 이상 업데이트 되지 않는다는 전제이므로
- 음의 가중치가 있는 그래프에서 사용이 불가능

최단 경로 JUNGOL 1941

- 문제
 - 그래프가 입력으로 들어왔을 때 1번 정점부터 N번 정점으로 이동할 경우의 최단 경로를 구하는 프로그램을 작성하라.
- 풀이
 - 한 정점에서 다른 정점으로의 최단 경로를 구하는 문제 -> 다익스트라 알고리즘
 - $N \leq 20,000$, $M \leq 100,000$ 이므로 선형 탐색을 한다면 시간초과가 날 것
 - 우선순위 큐를 사용해 풀어보자

```

// JO 1941 최단 경로
#include <iostream>
#include <queue>
#include <utility>
#include <vector>

#define SIZ 20001
#define INF 987654321
using namespace std;

vector<pair<int, int>> edges[SIZ];

int d[SIZ];
int N, M;

void dijkstra() {
    // 현재 거리가 가장 낮은 노드 먼저 살펴봄
    // first : 현재 가중치 second : 노드
    priority_queue<pair<int, int> > pq;

    // 최단 거리 초기화
    for(int i = 1; i ≤ N; i++)
        d[i] = INF;
    d[1] = 0;

    pq.push(make_pair(0, 1));

    while(!pq.empty()) {
        pair<int, int> cur = pq.top(); pq.pop();

        int w = -cur.first;
        int node = cur.second;

        // 만약 현재 거리가 d에 저장된 거리보다 크다면 더 탐색 x
        if(d[node] < w) continue;

        // 가능한 다음 노드들에 대하여
        for(auto nxt : edges[node]) {
            int n_node = nxt.first;
            int n_w = w + nxt.second;
            // 만약 d에 저장된 거리보다 크다면 pq에 추가 x
            if(d[n_node] ≤ n_w)
                continue;
            d[n_node] = n_w;
            pq.push(make_pair(-n_w, n_node));
        }
    }
}

```

```

int main() {
    cin >> N >> M;

    while(M--) {
        int a, b, c;
        cin >> a >> b >> c;
        edges[a].push_back(make_pair(b, c));
    }

    dijkstra();
    cout << d[N];
}

```


플로이드-워셜 알고리즘

- 플로이드-워셜 알고리즘이란?
 - 모든 정점 쌍 a-b의 최단 경로를 구하는 알고리즘
 - $\text{dist}[k][a][b]$ = 1 ~ k번째 정점을 중간 정점으로 건널 때 a에서 b로가는 최단 경로
 - 이때 k번째 정점을 거치거나, 1 ~ k - 1번째 정점만 거치는 경우 중 최솟값이 $\text{dist}[k][a][b]$
 - 따라서 $\text{dist}[k][a][b] = \min(\text{dist}[k-1][a][b], \text{dist}[k-1][a][k] + \text{dist}[k-1][k][b])$
 - 이때 $\text{dist}[k]$ 는 $\text{dist}[k-1]$ 만 참조하므로 2차원 배열로 표현 가능

플로이드-워셜 알고리즘

```
// 만약 a와 b를 연결하는 간선이 존재한다면 dist[a][b] = w로 초기화
// 그러한 간선이 없다면 dist[a][b] = INF로 초기화
// dist[a][a] = 0

for(int k = 1; k ≤ N; k++) {
    for(int a = 1; a ≤ N; a++) {
        for(int b = 1; b ≤ N; b++)
            dist[a][b] = min(dist[a][b], dist[a][k] + dist[k][b]);
    }
}
```

플로이드-워셜 알고리즘

- 시간 복잡도
 - 두 정점 a, b 를 고르는 방법 $\rightarrow N^2$
 - 중간 정점 k 를 고르는 방법 $\rightarrow N$
 - 3중 반복문을 수행하므로 $O(N^3)$
- 정점 수가 작고, 모든 정점 쌍의 최단 거리를 구할 때 사용

저울 JUNGOL 2753

- 문제

- 무게가 서로 다른 물건 N 개와, 일부 물건 쌍의 무게 비교 결과가 주어진다.

이때 각 물건에 대해 비교 결과를 알 수 없는 물건의 개수를 출력하시오.

저울 JUNGOL 2753

- 문제

- 무게가 서로 다른 물건 N개와, 일부 물건 쌍의 무게 비교 결과가 주어진다.

이때 각 물건에 대해 비교 결과를 알 수 없는 물건의 개수를 출력하시오.

- 풀이

- a가 k보다 무겁고, k가 b보다 무겁다면 a는 b보다 무겁다
 - $\text{heavier}[a][b] = \text{"a가 b보다 무겁다면 true, 모른다면 false"}$ 라고 했을 때
 - $\text{heavier}[a][b] = \text{heavier}[a][k] \ \&\& \ \text{heavier}[k][b]$
 - 플로이드 워셜의 점화식을 수정해 풀면 $O(N^3)$ 에 풀이 가능

```

// JO 2753 저울
#include <iostream>

using namespace std;

bool heavier[101][101];
int ans[101];

int main() {
    int N, M;
    cin >> N >> M;

    while(M--) {
        int a, b;
        cin >> a >> b;
        heavier[a][b] = true;
    }

    for(int k = 1; k ≤ N; k++) {
        for(int a = 1; a ≤ N; a++) {
            for(int b = 1; b ≤ N; b++) {
                // 만약 a가 k보다 무겁고, k가 b보다 무겁다면 a는 b보다 무거움을 알 수 있다
                if(heavier[a][k] && heavier[k][b])
                    heavier[a][b] = true;
            }
        }
    }

    for(int a = 1; a ≤ N; a++) {
        for(int b = 1; b ≤ N; b++) {
            if(a == b) continue;
            // 만약 a가 b보다 큰지도 모르고, b가 a보다 큰지도 모른다면 비교 결과를 알 수 없다
            if(!heavier[a][b] && !heavier[b][a])
                ans[a] += 1;
        }
    }

    for(int i = 1; i ≤ N; i++)
        cout << ans[i] << endl;
}

```

벨만-포드 알고리즘

- 벨만 포드 알고리즘이란 ?

- 음의 가중치가 포함된 그래프에서 시작 정점 s 로부터 다른 모든 정점까지의 최단 거리를 구하는 알고리즘
- 기존의 다익스트라 알고리즘은 간선의 가중치가 0 이상일 때만 사용 가능
 - ※ 이미 최단 거리를 구한 집합 S 에서, 가중치가 음수인 간선을 통해 최단 거리가 갱신될 수 있다
 - ※ 따라서 그리디한 선택이 불가능하다
- 시작 정점 s 에서 다른 모든 정점까지의 최단 거리를 구한다고 가정해보자
- 1) 최단 경로는 같은 정점을 두 번 이상 지나지 않는다.
- 2) 음수 사이클이 없다면, 최단 경로에 포함되는 간선 수는 최대 $V - 1$ 이다.
- 3) 즉 모든 간선 $w(u, v)$ 에 대해 $d[v] = \min(d[v], d[u] + w(u, v))$ 연산을 $V - 1$ 번 수행한다면 최단 거리가 갱신된다.

벨만-포드 알고리즘

- 음수 사이클 판단

- 만약 경로 갱신을 $V - 1$ 번 한 상태에서, 한번 더 모든 간선에 대하여 $d[v] = \min(d[v], d[u] + w(u, v))$ 연산을 수행했을 때 경로가 갱신된다면 음수 사이클이 발생했음을 알 수 있다

※ 즉 적어도 한 노드를 두 번 이상 방문하면서 경로의 가중치 합이 줄어듦 -> 음수 간선이 존재함

- 만약 dist 배열을 0으로 초기화 한다면, 모든 노드를 시작 노드로 했을 때의 음수 사이클의 존재 여부를 확인할 수 있음

- 시간 복잡도

- 각 간선을 $V - 1$ 번 업데이트 하므로 $O(VE)$


```

// 노드 a에서 b로 가는 가중치가 w인 간선
struct edge {
    int a, b, w;
    edge(int a, int b, int w) {
        this->a = a;
        this->b = b;
        this->w = w;
    }
};

// 노드 번호가 1 ~ V일 때, 1번 노드에서 다른 노드로의 최단거리 구하기
bool bellmanFord(int V, vector<edge> edges) {
    bool hasCycle = false;
    vector<int> dist(V + 1, INF);
    dist[1] = 0;

    // 경로 갱신 연산을 총 V - 1번 수행
    for(int i = 0; i < V - 1; i++)
        for(edge e : edges)
            dist[e.b] = dist[e.a] + e.w;
    // 만약 한번 더 갱신했을 때 dist가 짧아진다면
    // 1에서 출발하는 경로 중 음수 사이클이 존재함
    for(edge e : edges)
        if(dist[e.b] > dist[e.a] + e.w)
            hasCycle = true;

    return hasCycle;
}

```

```

// 노드 a에서 b로 가는 가중치가 w인 간선
struct edge {
    int a, b, w;
    edge(int a, int b, int w) {
        this->a = a;
        this->b = b;
        this->w = w;
    }
};

// 노드 번호가 1 ~ V일 때, 1번 노드에서 다른 노드로의 최단거리 구하기
bool bellmanFord(int V, vector<edge> edges) {
    bool hasCycle = false;
    // 모든 정점의 거리를 0으로 초기화한다면
    // 임의의 정점에서 출발하는 간선 중 음수 사이클이 존재하는지 확인 가능
    vector<int> dist(V + 1, 0);

    // 경로 갱신 연산을 총 V - 1번 수행
    for(int i = 0; i < V - 1; i++)
        for(edge e : edges)
            dist[e.b] = dist[e.a] + e.w;
    // 만약 한번 더 갱신했을 때 dist가 짧아진다면
    // 음수 사이클이 존재함
    for(edge e : edges)
        if(dist[e.b] > dist[e.a] + e.w)
            hasCycle = true;

    return hasCycle;
}

```

블랙홀 JUNGOL 2223

- 문제
 - 농지 곳곳에 블랙홀과 평범한 이동 경로가 있다. 블랙홀을 타면 다른 농지로 이동할 수 있는데, 이 경우 시간이 거꾸로 흘러갈 수 있다.
 - 한 농지에서 출발해 다시 출발했던 영역으로 돌아왔을 때 시간이 과거가 되도록 할 수 있는지 판단하는 프로그램을 작성하라

블랙홀 JUNGOL 2223

- 문제
 - 농지 곳곳에 블랙홀과 평범한 이동 경로가 있다. 블랙홀을 타면 다른 농지로 이동할 수 있는데, 이 경우 시간이 거꾸로 흘러갈 수 있다.
 - 한 농지에서 출발해 다시 출발했던 영역으로 돌아왔을 때 시간이 과거가 되도록 할 수 있는지 판단하는 프로그램을 작성하라
- 풀이
 - 블랙홀을 포함한 모든 이동 경로를 $V - 1$ 번 업데이트 하고, 다시 최단 경로를 갱신하였을 때 그 값이 줄어든다면 음수 사이클이 발생함 -> 과거로 돌아갈 수 있음
 - 임의의 정점에서 출발했을 때 음수 간선이 존재하는지만 확인하면 되므로, dist를 잘 초기화해보자

최단 경로 알고리즘 비교

알고리즘/비교	시작 정점 수	시간복잡도	음수 간선 가능	주 목적
다익스트라	1	$O((V + E) \log V)$	불가능	하나의 정점에서 최단 경로 계산
플로이드-워셜	모든 노드	$O(N^3)$	가능	모든 정점에서 최단 경로 계산
벨만-포드	1	$O(VE)$	가능	최단 경로 계산 및 음수 사이클 존재 여부 판단

```
// JO 2223 블랙홀
#include <iostream>
#include <vector>
#include <string.h>

using namespace std;

#define INF 0x3f3f3f3f

struct edge {
    int a, b, w;
    edge(int a, int b, int w) {
        this->a = a;
        this->b = b;
        this->w = w;
    }
};
```

```
int main() {
    int F, dist[501];
    cin >> F;

    while(F--) {
        memset(dist, 0, sizeof(dist));

        int N, M, B;
        bool possible = false;

        cin >> N >> M >> B;

        vector<edge> edges;

        while(M--) {
            int S, E, T;
            cin >> S >> E >> T;
            edges.push_back(edge(S, E, T));
            edges.push_back(edge(E, S, T));
        }

        while(B--) {
            int S, E, T;
            cin >> S >> E >> T;
            edges.push_back(edge(S, E, -T));
        }

        // N - 1번 업데이트하기
        for(int V = 0; V < N - 1; V++) {
            for(edge e : edges)
                dist[e.b] = min(dist[e.b], dist[e.a] + e.w);
        }

        // 만약 한번 더 돌렸는데 dist가 갱신된다면 사이클이 발생
        for(edge e : edges) {
            if(dist[e.b] > dist[e.a] + e.w)
                possible = true;
        }

        if(possible) cout << "YES\n";
        else cout << "NO\n";
    }
}
```

경로 추적

- 모든 정점 최단 경로
 - 플로이드-워셜 알고리즘에서 최단 경로를 추적
 - $\text{nxt}[a][b]$: a에서 b로 가는 최단 경로 중 a 다음에 오는 정점 번호
 - 라고 정의하면 dist 배열 업데이트 중에서 nxt 배열 업데이트 가능
 - 이후 nxt 배열을 추적하며 최단 경로 확인 가능

```

int N;
int w[101][101];    // w[a][b] : a와 b를 잇는 간선의 가중치
int dist[101][101]; // dist[a][b] : a에서 b로 가는 최단 경로
int nxt[101][101];  // nxt[a][b] : a에서 b로 가는 최단 경로중 a다음에 올 정점
void floyd_warshall() {

    // 초기화
    for(int a = 1; a ≤ N; a++) {
        for(int b = 1; b ≤ N; b++) {
            if(a == b) {
                dist[a][b] = 0;
                nxt[a][b] = a;
            }
            else if(w[a][b] ≠ INF) {
                dist[a][b] = w[a][b];
                nxt[a][b] = b;
            }
            else {
                dist[a][b] = INF;
                nxt[a][b] = -1;
            }
        }
    }

    for(int k = 1; k ≤ N; k++)
        for(int a = 1; a ≤ N; a++)
            for(int b = 1; b ≤ N; b++)
                // dist 배열 업데이트 하면서 nxt 배열 업데이트
                if(dist[a][b] > dist[a][k] + dist[k][b]) {
                    dist[a][b] = dist[a][k] + dist[k][b];
                    nxt[a][b] = nxt[a][k];
                }
}

```

```

// S에서 D로 가는 최단 경로 리턴
vector<int> getPath(int S, int D) {
    vector<int> path;

    // 경로가 없는 경우 -1 리턴
    if(nxt[S][D] == -1)
        return path;

    path.push_back(S);

    while(S ≠ D) {
        // S는 다시 S→D를 잇는 경로에서 S 다음 정점이 됨
        S = nxt[S][D];
        path.push_back(S);
    }

    return path;
}

```

경로 추적

- 단일 정점 최단 경로

- 다익스트라, 벨만 포드 알고리즘은 시작 정점이 하나이므로 1차원 배열로 경로 추적 가능
- $prev[d]$: 시작 정점 S에서 d로 올 때, d 바로 직전의 정점
- 라고 정의하면 최단 거리를 업데이트 할 때 같이 갱신 가능


```

int N;
vector<int> pre, dist;
vector<pair<int, int>> edges[101];
vector<int> dijkstra() {

    dist.resize(N + 1, INF);
    pre.resize(N + 1, -1);

    priority_queue <
        pair<int, int>,
        vector<pair<int, int>>,
        greater<pair<int, int>>
    > pq;

    dist[1] = 0;
    pq.push(make_pair(0, 1));

    while(!pq.empty()) {
        pair<int, int> cur = pq.top(); pq.pop();
        int cur_node = cur.second;
        int cur_weight = cur.first;

        if(dist[cur_node] ≤ cur_weight)
            continue;

        for(pair<int, int> nxt : edges[cur_node]) {
            int nxt_node = nxt.second;
            int nxt_weight = nxt.first + cur_weight;

            if(dist[nxt_node] ≤ nxt_weight)
                continue;
            // 최단 경로를 업데이트 하면서 pre 배열도 업데이트
            dist[nxt_node] = nxt_weight;
            pre[nxt_node] = cur_node;
            pq.push(make_pair(nxt_weight, nxt_node));
        }
    }

    return dist;
};

```

```

vector<int> getPath(int D) {
    vector<int> path;

    if(dist[D] == INF)
        return path;

    while(D ≠ -1) {
        path.push_back(D);
        D = pre[D];
    }
    // 뒤에서부터 추적했으므로 reverse
    reverse(path.begin(), path.end());
    return path;
}

```